

从构建到依赖隔离

Lab 01-05 完整复习笔记

结合实验现象、你的回答与薄弱点重新讲解

适用环境：Windows · JDK 11 · Maven Wrapper 3.9.9

实验目录：ratis-labs（与 Ratis 源码仓库分离）

版本：2026-07-19 用途：后续巩固、排错和阅读 Apache Ratis 构建配置

阅读说明

这不是把五个 README 简单拼接起来，而是按你的实际学习轨迹重组。你已经能完成命令与实验，当前最需要加强的是：同一术语在“项目模型、构建过程、打包结果、JVM 运行”四个层次中的边界。很多疑问并非命令不会用，而是把不同层次混在了一起。

层次	核心问题	相关文件
项目模型	项目是谁、依赖谁、如何构建？	pom.xml、GAV、dependency、plugin
一次构建	这次命令准备构建哪些项目？顺序是什么？	Reactor、-pl、-am、生命周期
构建产物	target 里生成了什么？JAR 内到底有什么？	classes、thin JAR、fat JAR
运行时	JVM 最终从哪里加载哪个 class？	classpath、类加载、NoSuchMethodError

建议复习方法

第一次顺序通读；第二次边读边运行每章的“最小验证命令”；第三次只看每章末尾的检查点并口述答案。

目录

阅读说明	2
目录	3
1. 先建立一张 Maven 总地图	5
1.1 一句话模型	5
1.2 从命令到结果	5
2. Lab 01: POM、目录、生命周期与 JAR	5
2.1 Maven 怎样找到源码	5
2.2 GAV: 构件的地址	6
2.3 生命周期不是一条命令列表	6
2.4 为什么 `java -cp` 有时接目录, 有时接 JAR	6
2.5 普通 JAR、可执行 JAR 先分开	7
3. Lab 02: 依赖、测试、本地仓库	7
3.1 dependency 与 plugin 是两条线	7
3.2 scope 决定依赖在哪些 classpath 可见	7
3.3 直接依赖与传递依赖	7
3.4 package 与 install 的边界	8
3.5 Maven 找依赖的典型顺序	8
4. Lab 03: 多模块、Parent 与当前 Reactor	8
4.1 Parent 与 Aggregator 是两个关系	8
4.2 什么是 Reactor, 什么是 “当前 Reactor”	8
4.3 `-pl` 和 `-am` 到底改变了什么	9
4.4 你的核心疑问: 没有 library JAR, 为何加 `-am` 就成功	9
5. Lab 04: 依赖仲裁与 JVM 类冲突	10
5.1 实验结构	10
5.2 Maven 怎样判断 “同一依赖的多版本冲突”	10
5.3 仲裁规则: nearest definition	10
5.4 为什么各模块编译成功, 应用却运行失败	10
5.5 四类常见错误要分清	11
5.6 两个不同坐标的 JAR 内有同名 class, Maven 怎么办	11

6. Lab 05: Thin JAR、Fat JAR、Shade 与 Relocation	11
6.1 先用两个维度看 JAR	11
6.2 Thin/ordinary JAR	12
6.3 Fat/Uber JAR	12
6.4 为什么 Fat JAR 仍没解决 Lab04	12
6.5 Relocation 才是版本隔离	12
6.6 为什么 app-safe 能有两个 TextFormatter	13
6.7 映射到 Apache Ratis	13
6.8 `skipShade` 为什么不是通用开关	13
7. 一套可复用的 Maven 排错流程	13
7.1 先判断问题发生在哪一层	13
7.2 六步证据链	13
7.3 Windows 常用命令速查	14
8. 读 POM 时应该按什么顺序	14
8.1 七步阅读法	14
8.2 `pluginManagement` 与 `plugins`	15
8.3 属性不是功能	15
9. 你的薄弱点与针对性巩固	15
9.1 已经掌握的部分	15
9.2 仍需反复强化的四条边界	15
9.3 复习时必须先预测	16
10. 自测题	16
第一组：必须口述	16
第二组：动手证明	16
11. 自测答案要点	17
12. 术语速查	17
资料来源与实验对应	18

1. 先建立一张 Maven 总地图

1.1 一句话模型

必须能脱口而出

Maven 读取 POM，按约定定位源码和资源，解析依赖，然后按生命周期调用插件，把输入变成 target 中的构建产物。

这句话包含四个角色：

POM	声明项目坐标、依赖、插件和构建配置	不是执行脚本；XML 标签本身不会编译代码
生命周期	规定阶段顺序	不是具体工具实现
插件	真正执行编译、测试、打包等工作	不是业务依赖
仓库	按坐标保存和查找构件	不是当前 Reactor

1.2 从命令到结果

```
.\mvnw.cmd -f .\lab-01-hello-maven\pom.xml clean package
```

命令选择 POM

- > Maven 建立项目模型
- > 解析插件与依赖
- > clean 生命周期删除 target
- > default 生命周期执行到 package
- > 插件编译、测试、打包
- > target/hello-maven-1.0-SNAPSHOT.jar

`mvnw.cmd` 是 Maven Wrapper 的 Windows 启动脚本。即使机器上已经安装 `mvn`，仍推荐在项目中使用 Wrapper，因为它把 Maven 版本固定为项目所验证的版本。系统 `mvn` 适合个人临时操作；Wrapper 更适合可复现构建、团队协作和 CI。

你的一个关键薄弱点

看到 BUILD SUCCESS 时，要继续追问：成功的是哪个目标？生成了什么？JAR 里有什么？运行 classpath 又是什么？“构建成功”从不自动回答后三个问题。

2. Lab 01：POM、目录、生命周期与 JAR

2.1 Maven 怎样找到源码

Maven 依靠“约定优于配置”。生产源码根目录默认为 `src/main/java`。Java 文件中的 package 决定它在该根目录下的相对路径。

```
package study.ratis.lab;
```

```
src/main/java/study/ratis/lab/HelloMaven.java
^ package 名逐段对应目录
```

Maven 并不是遍历磁盘猜测源码，也不是根据类名去整个项目搜索。遵守标准目录后，POM 无需反复声明 sourceDirectory。

2.2 GAV: 构件的地址

`groupId:artifactId:version` 合称 GAV。Lab01 的坐标是 `study.ratis:hello-maven:1.0-SNAPSHOT`。坐标描述的是 Maven 构件身份，不是 Java 类的身份。这个区别会在 Lab04 成为核心。

属性	值	说明
groupId	study.ratis	组织或命名空间
artifactId	hello-maven	构件名称
version	1.0-SNAPSHOT	开发中的可变版本
packaging	jar (默认)	产物类型; 不写时默认 jar

2.3 生命周期不是一条命令列表

Maven 有多个生命周期。常用的 default 生命周期阶段依次包括 validate、compile、test、package、verify、install、deploy。执行后面的阶段，会先完成它前面的阶段。`package` 因此会包含编译和测试；但 `clean` 属于独立的 clean 生命周期。

阶段	生命周期	生命周期中的输出文件
validate	验证项目模型	通常还没有 .class 或 JAR
compile	编译生产代码	target/classes/.../*.class
test	编译并运行测试	target/test-classes、surefire-reports
package	生成当前模块产物	target/*.jar
install	把产物和 POM 写入本地仓库	%USERPROFILE%/.m2/repository/...
clean	删除构建目录	target 被删除

精确表述

不要把顺序简写成 validate -> compile -> test -> package 后误以为只有这四个阶段。中间还有资源处理、testCompile 等阶段，只是日常常用目标用这几个名字概括。

2.4 为什么 `java -cp` 有时接目录，有时接 JAR

Classpath 的每一项是一个“类路径根”。JVM 把全限定类名 `study.ratis.lab.HelloMaven` 转成相对路径 `study/ratis/lab/HelloMaven.class`，然后依次到每个根中寻找。根既可以是目录，也可以是 JAR。

```
# compile 后: 目录是类路径根
java -cp .\lab-01-hello-maven\target\classes study.ratis.lab.HelloMaven

# package 后: JAR 是类路径根
java -cp .\lab-01-hello-maven\target\hello-maven-1.0-SNAPSHOT.jar `
study.ratis.lab.HelloMaven`
```

答案

两条命令的查找逻辑完全相同，只是 `.class` 的容器不同：第一条放在目录树里，第二条放在 ZIP 格式的 JAR 里。

2.5 普通 JAR、可执行 JAR 先分开

普通 JAR 可以放入 classpath，但不一定能 `java -jar`。可执行 JAR 的 manifest 中声明了 `Main-Class`。此处还不能把“可执行”理解为“依赖齐全”；Lab05 会把这两个维度彻底分开。

Lab01 最小验证

```
.\mvnw.cmd -f .\lab-01-hello-maven\pom.xml clean compile
Get-ChildItem .\lab-01-hello-maven\target\classes -Recurse

.\mvnw.cmd -f .\lab-01-hello-maven\pom.xml package
jar tf .\lab-01-hello-maven\target\hello-maven-1.0-SNAPSHOT.jar
```

3. Lab 02：依赖、测试、本地仓库

3.1 dependency 与 plugin 是两条线

类型	作用范围	用途	例子
dependency	项目的编译/测试/运行 classpath	业务代码需要的库	JUnit、Guava、Netty
plugin	Maven 的构建执行过程	完成某个构建动作	compiler、surefire、jar、shade

JUnit 在 Lab02 中是 test scope 的 dependency；Surefire 是负责发现并运行测试的 plugin。一个提供 API 和引擎，另一个在 Maven 生命周期中执行测试。

3.2 scope 决定依赖在哪些 classpath 可见

scope	compile	runtime	test	provided	系统库
compile	是	是	是	通常是	普通业务库
provided	是	是	由容器提供	否	Servlet API
runtime	否	是	是	通常是	数据库驱动
test	否	是	否	否	JUnit

重要修正

“compile scope 参与打包”不等于 dependency 的 class 会被复制进普通 JAR。scope 决定依赖的可见性和传递性；普通 maven-jar-plugin 默认仍只打包当前模块自己的 classes/resources。

3.3 直接依赖与传递依赖

POM 直接声明 `junit-jupiter`，它又依赖 `junit-jupiter-api`、`junit-jupiter-engine` 等。前者是直接依赖，后者是传递依赖。使用依赖树可以看到来源、scope、被仲裁掉的版本。

```
.\mvnw.cmd -f .\lab-02-tests-and-repository\pom.xml dependency:tree
.\mvnw.cmd -f .\lab-02-tests-and-repository\pom.xml test
```

3.4 package 与 install 的边界

`package` 在当前项目的 target 中生成 JAR; `install` 在完成 package 后, 将 JAR 与 POM 复制到本地仓库, 并按坐标形成路径。

```
坐标: study.ratis:calculator:1.0-SNAPSHOT

%USERPROFILE%\m2\repository\study\ratis\calculator\1.0-SNAPSHOT\
calculator-1.0-SNAPSHOT.jar
calculator-1.0-SNAPSHOT.pom
```

本地仓库不是运行 classpath, 也不是当前 Reactor。它是 Maven 在多次独立构建之间复用构件的持久缓存/仓库。删除 target 不会删除本地仓库; clean 也不会清理 `.m2`。

3.5 Maven 找依赖的典型顺序

- 1 如果依赖来自当前多模块构建的 Reactor, 优先使用 Reactor 中对应项目的产物模型。
- 2 否则查本地仓库。
- 3 本地没有时, 再按 settings.xml 和 POM 配置访问远程仓库并下载到本地。

排错习惯

怀疑依赖来源时先运行 `dependency:tree`; 怀疑仓库文件时再查看 `.m2/repository`。不要只通过“磁盘上有没有某个 JAR”猜 Maven 此次构建用了什么。

4. Lab 03: 多模块、Parent 与当前 Reactor

4.1 Parent 与 Aggregator 是两个关系

关系	Parent 关系	Aggregator 关系
继承 (parent)	子 POM 的 ``	配置从哪里来: groupId、version、properties、pluginManagement 等
聚合 (modules)	聚合 POM 的 ``	一次构建包含哪些项目

同一个 POM 可以既是 parent 又是 aggregator, 但概念不能混用。子项目可以继承一个不聚合它的 parent; 聚合项目也可以聚合并不继承它的模块。

4.2 什么是 Reactor, 什么是“当前 Reactor”

Reactor 是 Maven 为“一次命令”收集到的项目集合, 以及根据模块依赖关系计算出的构建顺序。它是内存中的本次构建上下文, 不是目录、不是仓库、也不是长期存在的服务。命令结束, 当前 Reactor 就结束。

```
.\mvnw.cmd -f .\lab-03-multi-module\pom.xml package

当前 Reactor:
1. multi-module-parent
2. greeting-library
3. greeting-app
```


虽然 `` 中的书写顺序会参与项目收集，但 Maven 会依据项目间依赖拓扑安排构建：`greeting-app` 依赖 `greeting-library`，所以 library 必须先于 app。

4.3 `-pl` 和 `-am` 到底改变了什么

选项	含义/作用	对 Reactor 的影响
<code>-pl greeting-app</code>	projects list	只选择 app；其上游模块不自动加入
<code>-am</code>	also make required projects	把所选项目依赖的 Reactor 项目也加入

```
# 只选 app
.\mvnw.cmd -f .\lab-03-multi-module\pom.xml -pl greeting-app package

# 选 app，并把它依赖的 library 一起构建
.\mvnw.cmd -f .\lab-03-multi-module\pom.xml -pl greeting-app -am package
```

4.4 你的核心疑问：没有 library JAR，为何加 `-am` 就成功

因为 `-am` 把 `greeting-library` 加入当前 Reactor。Maven 先编译 library，并把它作为同一次构建中的项目依赖提供给 app 的编译 classpath。此过程不要求 library 预先 install 到 `.m2`。

现象	解释/原因
app 的 package 成功	编译时能从当前 Reactor 获得 library 的输出
app JAR 内没有 GreetingService.class	正常：普通 JAR 不复制 dependency 的 class
本地仓库没有 library 坐标	正常：执行的是 package，不是 install

一句话答案

`-am` 解决 “这次构建时去哪里拿上游模块”，不是 “把上游模块塞进 app JAR”，也不是 “把上游模块安装到本地仓库”。

把三个空间分开

当前 Reactor target	临时、属于本次命令：可供同次构建解析项目依赖
本地仓库 .m2	当前模块构建输出：classes、JAR、报告
	跨命令持久保存：需要 install 或远程下载

验证实验

- 1 先确保 greeting-library 没有 install 到本地仓库。
- 2 运行 ``-pl greeting-app -am clean package``，观察 Reactor Build Order。
- 3 用 ``jar tf`` 检查 app JAR，确认没有 GreetingService.class。
- 4 另开一次只针对 greeting-app POM 的独立构建；若本地仓库仍无 library，它无法再借用已经结束的 Reactor。

```
jar tf .\lab-03-multi-module\greeting-app\target\greeting-app-1.0-SNAPSHOT.jar `
| Select-String 'GreetingApp|GreetingService'
```

5. Lab 04: 依赖仲裁与 JVM 类冲突

5.1 实验结构

```
app-conflict
+-- consumer-a -> demo-thirdparty:1.0 -> TextFormatter.format()
+-- consumer-b -> demo-thirdparty:2.0 -> TextFormatter.upper()
```

v1 和 v2 的 Java 全限定类名相同：
study.ratis.lab.thirdparty.TextFormatter

5.2 Maven 怎样判断“同一依赖的多版本冲突”

Maven 依赖仲裁首先比较构件的冲突标识。对日常 JAR，可近似记为 `groupId:artifactId` 相同而 version 不同；更完整地说还会考虑 type 与 classifier。相同冲突标识的多个版本不能同时成为普通依赖图中的获胜版本，Maven 会进行 mediation。

回答你曾问的问题

是的，Maven 的版本仲裁依据 Maven 坐标身份，不会打开每个 JAR，逐个比较里面的 `.class` 是否重名。

5.3 仲裁规则：nearest definition

- 1 离当前项目依赖路径更近的版本优先。
- 2 路径深度相同时，通常先声明的依赖路径获胜。
- 3 dependencyManagement 可集中规定版本，但仍要结合实际依赖声明理解。

```
[INFO] +- consumer-a
[INFO] | \- demo-thirdparty:1.0
[INFO] \- consumer-b
[INFO]    \- (demo-thirdparty:2.0 - omitted for conflict with 1.0)
```

`omitted for conflict` 表示该节点在最终依赖图中因版本冲突未获胜。它不表示 JAR 损坏，也不表示编译器已经验证获胜版本兼容所有消费者。

5.4 为什么各模块编译成功，应用却运行失败

- 1 consumer-a 编译时使用 v1，`format()` 存在。
- 2 consumer-b 编译时使用 v2，`upper()` 存在。
- 3 app 编译时只直接调用 ConsumerA/ConsumerB 的公开方法，也能成功。
- 4 app 的最终运行 classpath 经仲裁只保留 v1 的 TextFormatter。
- 5 JVM 加载到了 TextFormatter，所以不是类缺失；但 v1 没有 `upper()`，链接时抛出 NoSuchMethodError。

你的原理解评价

你对“各自按自己的依赖编译，最终运行 classpath 只有一个版本，因此运行失败”的理解是正确的。需要补上的精确点是：冲突发生在 app 的依赖图/运行 classpath 形成时，而不是“打包动作天然只允许一个类”。普通 JAR 本身甚至不包含这些依赖。

5.5 四类常见错误要分清

异常	错误原因	解决方法
ClassNotFoundException	主动加载某类失败	类不在运行 classpath、类名错误
NoClassDefFoundError	运行时需要类定义不可用	依赖缺失、初始化失败
NoSuchMethodError	类存在，但运行时版本没有编译期调用的方法	二进制版本不兼容
Parameter/compile error	编译期 API 就不可用	编译 classpath 不对、源码错误

5.6 两个不同坐标的 JAR 内有同名 class，Maven 怎么办

如果两个 JAR 的坐标不同，Maven 通常把它们视为两个不同构件，二者都可以进入依赖图。即使都含有 `com.example.Duplicate.class`，Maven 默认也不会进行版本仲裁或自动报错。

到 JVM 运行时，同一个 ClassLoader 中一个全限定类名通常只能定义一次。Classpath 顺序靠前的定义往往先被加载，后面的同名 class 被遮蔽。结果可能是悄悄使用错误实现，也可能在稍后出现 NoSuchMethodError、LinkageError 或行为异常。

两套身份系统

Maven 看构件身份（坐标）；JVM 看类身份（ClassLoader + 全限定类名）。二者不是同一个冲突检测系统。

诊断命令

```
.\mvnw.cmd -f .\lab-04-dependency-conflict\pom.xml `
-pl app-conflict dependency:tree -Dverbose

jar tf path\to\first.jar | Select-String 'TextFormatter.class'
jar tf path\to\second.jar | Select-String 'TextFormatter.class'

# 观察 JVM 实际从哪个位置加载类 (JDK 11)
java -Xlog:class+load=info -cp "..." your.MainClass
```

6. Lab 05: Thin JAR、Fat JAR、Shade 与 Relocation

6.1 先用两个维度看 JAR

“能否 `java -jar`”与“是否包含依赖”是两个独立维度。把它们混成一条线，是你第一次学习 Lab05 时觉得跳跃的主要原因。

	是否可执行	是否包含依赖	说明
普通库 JAR	否	否	只能作为 classpath 构件
可执行 thin JAR	是	否	能找到 main，但可能因缺依赖失败
不可执行 fat JAR	否	是	依赖已合并，但仍无 `java -jar` 入口

类型	有 Main-Class	包含依赖 class	说明
可执行 fat JAR	是	是	通常可单文件启动

6.2 Thin/ordinary JAR

maven-jar-plugin 默认把当前模块的 `target/classes` 与资源放进 JAR，不会解压 dependency JAR。Lab05 的 thin-app 即使 manifest 有 Main-Class，单独运行仍会因缺 DemoLibrary 出现 NoClassDefFoundError。

```
# 单独运行：入口存在，但依赖不在 classpath
java -jar .\lab-05-shading-and-ratis\thin-app\target\thin-app-1.0-SNAPSHOT.jar

# 明确给两个 JAR：成功
java -cp "thin-app.jar;demo-library.jar" study.ratis.lab.packaging.ThinApp
```

6.3 Fat/Uber JAR

Fat JAR 与 Uber JAR 通常是同义词：把当前模块与依赖 JAR 的 class/resources 合并到一个大 JAR。Shade 插件可以完成这种合并。

`shade` 直译是“阴影、遮蔽、给……加阴影”。在 Java 构建语境中，它不是单纯的视觉翻译，而是指把依赖内容打进产物，并可通过改写包名让依赖隐藏在项目自己的命名空间中。

```
fat-app-1.0-SNAPSHOT.jar      <- shade 后的主产物
original-fat-app-1.0-SNAPSHOT.jar <- shade 前的普通 JAR 备份
```

6.4 为什么 Fat JAR 仍没解决 Lab04

Shade 在合并前拿到的是 Maven 已经解析、仲裁后的依赖集合。若 v1 获胜、v2 omitted，普通 Fat JAR 只会把 v1 放进去。它改变了 class 的存放位置和部署形态，却没有自动改变冲突类的全限定名。

固定顺序
依赖解析/仲裁 -> 编译与测试 -> package 阶段执行 shade -> 合并已选中的内容。Fat JAR 是仲裁结果的打包表现，不会凭空恢复被省略版本。

6.5 Relocation 才是版本隔离

Relocation 把其中一个依赖从原包名迁移到私有包名，使 JVM 将两者视为不同类。

```
原始 v1: study.ratis.lab.thirdparty.TextFormatter
迁移 v2: study.ratis.lab.internal.shaded.thirdparty.TextFormatter
```

Shade relocation 至少改写三处：

- JAR 中被迁移 class 的路径。
- class 文件内部记录的类名。
- 依赖该类的其他字节码中的符号引用，例如 ConsumerBShaded 对 TextFormatter.upper() 的调用。

因此源码仍然 `import study.ratis.lab.thirdparty.TextFormatter` 并不矛盾：源码先按原始 v2 编译；package 阶段 Shade 再改写生成的字节码。

```
javap -classpath .\lab-05-shading-and-ratis\consumer-b-shaded\target\consumer-b-shaded-1.0-SNAPSHOT.jar `
-c study.ratis.lab.consumerb.ConsumerBShaded

# 应看到 relocated 引用:
study/ratis/lab/internal/shaded/thirdparty/TextFormatter.upper
```

6.6 为什么 app-safe 能有两个 TextFormatter

它们的简单类名相同，但全限定类名已经不同，因此对 JVM 是两个不同类。v1 保持原包，v2 被迁移到 internal.shaded 包；各自消费者的字节码指向各自版本。

修正一个表述

不是 Maven 因 relocation 后“把它们认为是两个不同依赖”才允许共存；更准确地说，Shade 把 v2 的类身份与引用改写了，最终 JAR 中两个全限定类名不同，JVM 因而可以同时加载。

6.7 映射到 Apache Ratis

Ratis 可能与宿主系统同时依赖不同版本的 Netty、gRPC、Protobuf。如果都暴露原始包名，宿主应用最终 classpath 可能重演 Lab04。Ratis 因而使用独立 ThirdParty 构件，将相关类迁移到 `org.apache.ratis.thirdparty...` 命名空间。

```
com.google.protobuf -> org.apache.ratis.thirdparty.com.google.protobuf
io.grpc             -> org.apache.ratis.thirdparty.io.grpc
io.netty            -> org.apache.ratis.thirdparty.io.netty
```

独立构件的好处是隔离用户 classpath、集中升级和许可证管理、避免每次构建核心 Ratis 都重复执行大规模 shading；代价包括产物更大、调试包名变化、资源合并更复杂。

6.8 `skipShade` 为什么不是通用开关

`-DskipShade=true` 只是设置名为 skipShade 的 Maven 属性。只有某个 POM 或插件配置显式读取它（例如 `\${skipShade}`）时才有效。它不是 Maven 内置参数；当前 Ratis 核心构建若没有消费该属性，传入它不会产生作用。

7. 一套可复用的 Maven 排错流程

7.1 先判断问题发生在哪一层

POM 无法读取、模块不存在	项目模型、路径、parent/modules
编译找不到类	compile classpath、scope、Reactor、仓库
测试没运行/失败	test scope、Surefire、测试命名和报告
package 成功但 java -jar 失败	Main-Class、JAR 内容、运行 classpath
NoSuchMethodError	编译与运行依赖版本是否一致、dependency:tree
Fat JAR 中类仍冲突	仲裁结果、relocation、重复资源

7.2 六步证据链

- 1 确认正在使用的 JDK/Maven: ``mvnw.cmd -version``。
- 2 确认目标 POM 与模块集合: 查看 ``-f``、modules 和 Reactor Build Order。
- 3 查看有效依赖: ``dependency:tree -Dverbose``。
- 4 查看构建阶段输出: target/classes、target/test-classes、Surefire 报告。
- 5 查看 JAR 真内容: ``jar tf``, 不要根据文件名推断。
- 6 查看字节码/类加载: ``javap -c``、JDK 11 的 ``-Xlog:class+load=info``。

7.3 Windows 常用命令速查

```
# 固定 Maven 与 JDK 版本
.\mvnw.cmd -version

# 构建指定 POM
.\mvnw.cmd -f .\path\to\pom.xml clean package

# 多模块: 选模块并构建上游依赖
.\mvnw.cmd -f .\parent\pom.xml -pl module-name -am clean package

# 依赖树
.\mvnw.cmd -f .\pom.xml dependency:tree -Dverbose

# 看 JAR 内容
jar tf .\target\app.jar | Select-String 'SomeClass'

# 看 manifest
jar xf .\target\app.jar META-INF/MANIFEST.MF
Get-Content .\META-INF\MANIFEST.MF

# 看字节码调用目标
javap -classpath .\target\app.jar -c com.example.App
```

PowerShell 参数提示

对包含 ``-D...`` 的参数, 若 PowerShell 或插件出现解析歧义, 可以把整个参数写成字符串, 例如 ``"-Dexec.mainClass=com.example.App"``。但不要用引号掩盖拼写错误。

8. 读 POM 时应该按什么顺序

8.1 七步阅读法

- 1 看 coordinates: 当前项目是谁, packaging 是什么。
- 2 看 parent: 哪些 GAV、properties、dependencyManagement、pluginManagement 可能继承而来。
- 3 看 modules: 这个 POM 是否聚合其他项目。
- 4 看 properties: JDK、编码、依赖与插件版本属性。
- 5 看 dependencies: 业务 classpath 上需要什么、scope 是什么。
- 6 看 build/plugins: 实际绑定了哪些插件与 phase/goal。
- 7 最后看 profiles 和命令行 ``-D``: 哪些配置只有满足条件才生效。

8.2 `pluginManagement` 与 `plugins`

位置	作用
build/pluginManagement	集中提供默认版本和配置，本身通常不会让插件自动执行
build/plugins	当前项目实际声明使用插件；生命周期绑定或 execution 才会执行 goal

Lab03 的父 POM 在 pluginManagement 中管理 exec-maven-plugin 版本；greeting-app 在 plugins 中实际声明它。Lab05 的子模块把 shade goal 绑定到 package，package 时插件才会执行。

8.3 属性不是功能

```
<properties>
  <skipShade>true</skipShade>
</properties>
```

只有被其他配置引用才产生行为：

```
<configuration>
  <skip>${skipShade}</skip>
</configuration>
```

通用规律

`-Dname=value` 只是给 Maven 模型增加/覆盖一个属性。属性名看起来再像开关，也必须有 POM、插件或代码读取它。

9. 你的薄弱点与针对性巩固

9.1 已经掌握的部分

- 能正确区分 parent 继承与 modules 聚合。
- 能解释 Reactor 按依赖关系构建，以及 `-pl`/`-am` 的基本作用。
- 能根据 dependency:tree 与 NoSuchMethodError 还原编译成功、运行失败的链路。
- 能区分 thin JAR、可执行 JAR、fat JAR，并理解 relocation 会改写字节码引用。
- 能把 Ratis ThirdParty 的设计映射回实验中的依赖隔离问题。

9.2 仍需反复强化的四条边界

现象	本质
构建成功 vs 可独立运行	package 只说明目标阶段成功；运行仍取决于入口和 classpath
Reactor vs 本地仓库	Reactor 只活在本次命令；install 才把构件持久化到 .m2
依赖可见 vs 打进 JAR	scope 管可见性；普通 jar 插件不复制依赖
Maven 冲突 vs Java 类冲突	Maven 按坐标仲裁；JVM 按全限定类名加载

9.3 复习时必须先预测

每次运行命令前写下三个预测：Reactor 有哪些项目？每个模块 target 会新增什么？最终运行 classpath 需要哪些根？执行后只用 `dependency:tree`、`jar tf`、错误堆栈和 `javap` 验证。这个习惯比记住更多命令更重要。

10. 自测题

第一组：必须口述

- 1 `package` 为什么会执行 compile? `clean` 为什么不是 package 的前置阶段?
- 2 classpath 项为什么既可以是目录，也可以是 JAR?
- 3 test scope 的 JUnit 为什么能编译测试，却不会成为下游项目的普通传递依赖?
- 4 `-pl greeting-app -am package` 中，library 在哪里供 app 编译使用？它为什么不必已存在于本地仓库？
- 5 为什么 app JAR 里没有 GreetingService.class，package 仍能成功？
- 6 Maven 如何识别同一依赖的多个版本？两个不同坐标却含同名 class 时会怎样？
- 7 为什么 consumer-b 能编译通过，最终应用却抛 NoSuchMethodError?
- 8 有 Main-Class 是否等于 Fat JAR？Fat JAR 是否等于没有版本冲突？
- 9 Relocation 为什么不仅是把 class 文件移动到另一个目录？
- 10 `-DskipShade=true` 在什么条件下才有效？

第二组：动手证明

- 1 删除 Lab03 各模块 target，但不 install；用 `-pl/-am` 成功构建 app，并检查 app JAR。
- 2 运行 Lab04 dependency:tree，指出获胜版本、被省略版本和各自路径深度。
- 3 分别用 `jar tf` 检查 thin-app、fat-app、fat-conflict-app、app-safe 中的关键 class。
- 4 用 `javap -c` 找出 ConsumerBShaded 最终引用的 relocated 全限定类名。
- 5 尝试只运行 thin JAR，再手动补完整 classpath，解释两个结果。

11. 自测答案要点

- 1 生命周期阶段有顺序，执行 package 会完成 default 生命周期此前阶段；clean 属于另一个生命周期，只有命令显式写 clean 才执行。
- 2 JVM 把类名转换为相对路径，并在每个 classpath 根中查找；目录和 JAR 都能充当根。
- 3 scope 决定可见性和传递性；test 只用于测试编译/运行，不进入主运行 classpath，也不向下游普通传递。
- 4 library 被加入当前 Reactor，先构建并把输出提供给 app；当前 Reactor 可以解析同次构建项目，不需要预先 install。
- 5 普通 JAR 只含当前模块 class；dependency 用于编译 classpath 与打包是否复制依赖是两件事。
- 6 同冲突标识的不同版本按 nearest/先声明等规则仲裁；不同坐标通常都保留，同名 class 问题留给 classpath/JVM。
- 7 consumer-b 编译时看到 v2 的 upper；运行时仲裁后加载 v1，类存在但方法不存在。
- 8 Main-Class 只解决入口；Fat JAR 解决依赖合并；普通 Fat JAR 仍沿用 Maven 的仲裁结果。
- 9 Relocation 改写 class 路径、内部类名以及其他字节码的符号引用。
- 10 只有 POM/插件配置读取 skipShade 属性时才有效；它不是 Maven 内置开关。

12. 术语速查

术语	描述/说明
Artifact	Maven 构建或管理的构件，如 JAR、POM
GAV	groupId、artifactId、version
Lifecycle	有序阶段模型
Phase	生命周期中的阶段，如 compile、package
Goal	插件提供的具体任务，如 compiler:compile、shade:shade
Scope	依赖在哪些 classpath 可见及如何传递
Reactor	一次 Maven 命令收集并排序的项目集合
Repository	按坐标存储构件的位置，本地通常为 `.m2/repository`
Thin JAR	通常只含当前模块 class/resources 的普通 JAR
Fat/Uber JAR	合并当前模块与依赖内容的大 JAR
Shade	合并依赖，并可配合 relocation 隐藏/隔离依赖
Relocation	改写包名、类名与字节码引用，形成新的类身份
Mediation	同一依赖多个版本之间的仲裁
Classpath	JVM 搜索类和资源的一组根，顺序有意义

资料来源与实验对应

本笔记以你在 `ratis-labs` 中完成的 Lab01-Lab05 README、POM、源码、LEARNING_LOG 回答, 以及我们围绕 Reactor、`-am`、依赖仲裁、同名类、Thin/Fat JAR、Shade/Relocation、Ratis ThirdParty 和 skipShade 的问答为依据。

- lab-01-hello-maven: POM、标准目录、生命周期、target、JAR。
- lab-02-tests-and-repository: dependency、scope、测试、本地仓库。
- lab-03-multi-module: parent/modules、Reactor、`-pl/-am`。
- lab-04-dependency-conflict: 传递依赖、仲裁、classpath、NoSuchMethodError。
- lab-05-shading-and-ratis: Thin/Fat JAR、Shade、Relocation、Ratis ThirdParty。

最终记忆主线: POM 描述项目 -> Reactor 决定本次构建项目 -> 生命周期调用插件 -> 依赖形成各阶段 classpath -> target 产生构件 -> JVM 按运行 classpath 加载类。